

Project Deliverable - Scalable URL Shortener

Scalable URL Shortener with Distributed Orchestration

Virtualization Systems — Final Project Deliverable

Student: Seyedamin Ghazizadeh — 569071

Course: Virtualization Systems

Professors: Prof. Maria Fazio — Dr. Maurizio Giacobbe

Academic Year: 2025/2026

Date: June 12, 2026

Table of Contents

1. [Project Overview & Objectives](#)
 2. [Architecture Description](#)
 3. [Implementation Details](#)
 4. [Containerization Strategy](#)
 5. [Kubernetes Orchestration](#)
 6. [CI/CD Pipeline](#)
 7. [Performance & Scalability Considerations](#)
 8. [Virtualization Concepts Demonstrated](#)
 9. [Deployment Instructions](#)
 10. [Conclusion](#)
-

1. Project Overview & Objectives

1.1 Project Overview

This project implements a **production-grade, scalable URL shortener** service — functionally comparable to systems like Bitly or TinyURL — built entirely with a cloud-native, virtualization-first approach. The application accepts long URLs via a web interface or REST API, generates short, unique alphanumeric codes, and provides HTTP 302 redirects when the short code is accessed. It supports click tracking, statistics collection, bulk URL import, and in-memory caching for high-throughput performance.

1.2 Objectives

The primary objectives of this project are:

- **Demonstrate deep understanding of OS-level virtualization**

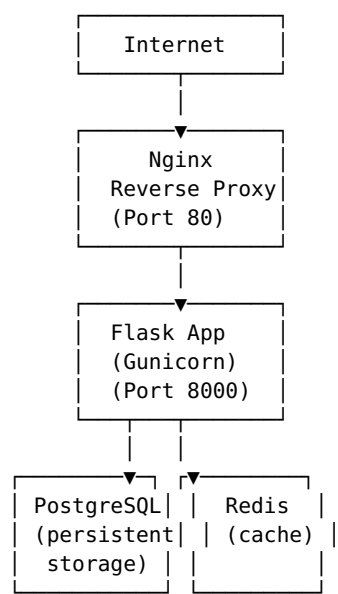
by designing and deploying a microservice-based application using Linux containers (Docker), cgroups resource constraints, and namespace-based isolation.

- **Implement container orchestration** by defining and deploying the system on Kubernetes with declarative YAML manifests covering deployments, services, autoscaling, configuration management, and ingress routing.
 - **Showcase production-grade CI/CD automation** by building a GitHub Actions pipeline that tests, builds, and deploys the application to a remote VPS via SSH-triggered webhook.
 - **Apply I/O optimization and concurrency patterns** through connection pooling, Redis caching, ThreadPoolExecutor-based parallel processing, and transaction-wrapped batch database inserts.
 - **Provide measurable performance characteristics** through defined resource limits, health probes, rate limiting, and horizontal autoscaling policies.
-

2. Architecture Description

2.1 Overall System Architecture

The system follows a **microservice-oriented architecture** decomposed into four distinct containerized services, each fulfilling a single responsibility:



2.2 Service Breakdown

Service	Role	Technology
Flask App	Core application server; handles URL shortening, redirects, stats, bulk import	Python/Flask, Gunicorn (WSGI)
Nginx	Reverse proxy, TLS termination,	nginx:1.25-alpine

	rate limiting, load balancing	
PostgreSQL	Persistent relational storage for URL mappings and click analytics	postgres:16-alpine
Redis	In-memory cache for fast short-code lookups (24h TTL)	redis:7-alpine

2.3 Container Isolation with Namespaces & cgroups

Each service runs inside its own Docker container, leveraging Linux kernel primitives for isolation:

- **Mount namespace (CLONE_NEWNS):** Each container has its own filesystem mount hierarchy. The Flask container sees only /app with the application code; PostgreSQL uses a dedicated volume (postgres_data) mounted at /var/lib/postgresql/data.
- **PID namespace (CLONE_NEWPID):** Processes inside one container cannot see or signal processes in another container. The Flask app sees PID 1 as the Gunicorn master process, while PostgreSQL's PID 1 is the postmaster.
- **Network namespace (CLONE_NEWNET):** Each container has its own network stack with a unique IP address on the Docker bridge network. Inter-service communication uses Docker Compose service DNS names (e.g., postgres:5432, redis:6379). Nginx binds to host port 80, acting as the sole ingress point.
- **User namespace:** The Flask Dockerfile creates a dedicated non-root app user and runs the application under that user, reducing the blast radius of a container breakout.
- **cgroups (resource constraints):** The docker-compose.yml defines explicit CPU and memory limits for every service using the deploy.resources block. These cgroup constraints ensure no single container can exhaust host resources, enforcing predictable performance isolation.

2.4 Nginx Reverse Proxy

Nginx serves as the system's single entry point, providing:

- **Reverse proxying:** All HTTP requests on port 80 are forwarded to the Flask backend at 127.0.0.1:8000 (sidecar pattern) or via upstream DNS in multi-pod deployments.
- **Rate limiting:** A shared memory zone (url_shortener_limit:10m) tracks client IPs and limits requests to 30 per second with a burst of 20. A connection limit zone restricts concurrent connections to 10 per IP.
- **Security headers:** X-Frame-Options, X-Content-Type-Options, and X-XSS-Protection are set on all responses.
- **Gzip compression:** Static assets and JSON responses are compressed on-the-fly with gzip level 6.
- **Health check passthrough:** The /api/health endpoint bypasses rate limiting for liveness/readiness probes.
- **Custom error pages:** JSON-formatted 429 (rate limit) and 502/503/504 (upstream unavailable) responses.

2.5 PostgreSQL (Persistent Storage)

PostgreSQL 16 Alpine provides reliable ACID-compliant storage. Configuration highlights:

- **Connection pooling:** The SQLAlchemy engine is configured with `pool_size=10`, `max_overflow=20`, and `pool_pre_ping=True` for connection health checks.
- **Persistent volume:** A named Docker volume (`postgres_data`) ensures database state survives container restarts.
- **Health check:** Docker healthcheck uses `pg_isready` to verify PostgreSQL readiness before the Flask app starts (configured via `depends_on.condition: service_healthy`).

2.6 Redis Caching Layer

Redis 7 Alpine provides an in-memory cache layer to reduce database load:

- **Cache-aside pattern:** On redirect, the app first checks Redis for the long URL (key format: `url:{short_code}`). On cache miss, it falls through to PostgreSQL and populates the cache.
 - **TTL:** 24-hour expiration (86,400 seconds) ensures stale entries are automatically evicted.
 - **Graceful degradation:** If Redis is unavailable, the `get_redis()` function returns `None` and all operations fall back to database queries. The app continues functioning without caching.
 - **Bulk population:** After bulk imports, Redis is populated using pipelined `SETEX` commands for efficiency.
-

3. Implementation Details

3.1 Flask Application Core

The application (`app.py`) is built with Flask 3.x and SQLAlchemy 3.x with the following key components:

3.1.1 Database Models

Two SQLAlchemy models define the relational schema:

- **URLMapping:** Stores `id`, `short_code` (unique, indexed), `long_url`, `click_count` (integer, default 0), and `created_at` timestamp. Has a one-to-many relationship with Click records.
- **Click:** Records individual redirect events with `url_mapping_id` (foreign key), `timestamp`, `user_agent`, `referrer`, and `ip_address`.

3.1.2 URL Shortening Logic

The `/shorten` endpoint:

1. Applies **rate limiting** (100 requests per 60-second sliding window per IP) using an in-memory dictionary.
2. Validates the incoming URL with a regex check and auto-prepends `https://` if no scheme is present.
3. Checks for **duplicate URLs** in the database and returns the existing short code if found, avoiding data duplication.
4. Generates a **6-character random alphanumeric short code** (using `random.choices` from `string.ascii_letters + string.digits`, giving $62^6 \approx 56$ billion possible codes).
5. Handles **collisions** by retrying generation until a unique code is found.
6. **Caches** the new mapping in Redis with a 24-hour TTL.
7. Returns the short URL and a created boolean flag.

3.1.3 Redirect with Click Tracking

The `/<short_code> redirect` endpoint:

1. Attempts a **Redis cache lookup** first (O(1) time complexity).
2. Falls back to a **database query** on cache miss, then populates the cache.
3. Records a **Click event** with the User-Agent, Referrer, and client IP address.
4. Atomically increments `click_count` on the URLMapping row using an SQL UPDATE statement.
5. Returns an HTTP **302 redirect** to the original long URL.

3.1.4 Statistics Endpoint

The `/stats/<short_code>` endpoint returns JSON with `total_clicks`, `recent_clicks_24h`, and `created_at`, enabling analytics visualization.

3.2 Bulk Import with ThreadPoolExecutor

The `/bulk-import` endpoint demonstrates **parallel processing** and **I/O optimization**:

1. **File parsing:** Accepts .csv files (via `csv.DictReader`) or plain text files (one URL per line, comments starting with `#` are ignored).
2. **Chunking:** The input rows are divided into chunks (`chunk_size = len(rows) // MAX_WORKERS` where `MAX_WORKERS` defaults to 4, configurable via the `BULK_IMPORT_WORKERS` environment variable).
3. **ThreadPoolExecutor:** Each chunk is submitted to a `ThreadPoolExecutor` with `max_workers=MAX_WORKERS`. Worker functions validate URLs, normalize them, and generate unique short codes while tracking a local set of codes to avoid intra-batch collisions.
4. **Result aggregation:** Completed futures are collected with `as_completed()` for non-blocking result processing.
5. **Deduplication:** Within-batch duplicates and existing database URLs are filtered out before insertion.
6. **Transaction-wrapped batch insert:** A single `db.session.bulk_insert_mappings()` call inserts all valid entries in one round-trip, wrapped in a transaction. On failure, `db.session.rollback()` prevents partial data.
7. **Redis pipeline:** After successful insert, a Redis pipeline populates the cache for all new entries in a single network round-trip.

3.3 Rate Limiting

Two layers of rate limiting protect the application:

- **Application layer (Flask):** In-memory sliding window per IP address. Default: 100 requests per 60-second window. The `_rate_store` dictionary is periodically cleaned of stale entries to prevent memory leaks.
- **Infrastructure layer (Nginx):** 30 requests/second with burst of 20, plus connection limiting to 10 concurrent connections per IP. Uses a 10MB shared memory zone (tracking ~160,000 IPs).

3.4 Click Tracking

Each redirect invocation creates a Click record capturing:

- `url_mapping_id` (foreign key to the URLMapping)
- `timestamp` (auto-generated UTC timestamp)
- `user_agent` (from the HTTP request header, truncated to 500 characters)
- `referrer` (from the HTTP Referer header)

- `ip_address` (from the remote address)

The click count is updated atomically on the URLMapping row to avoid race conditions under concurrent access.

3.5 Health Checks

Two health endpoints are provided:

- `/health` (legacy): Returns `{"status": "ok"}` with HTTP 200.
- `/api/health` (Kubernetes-ready): Checks database connectivity (`db.session.execute(db.select(func.now()))`) and Redis connectivity (`redis.ping()`). Returns `{"status": "healthy"|"degraded", "database": "up"|"down", "redis": "up"|"down", "timestamp": "..."} with HTTP 200 for healthy and 503 for degraded.`

4. Containerization Strategy

4.1 Dockerfile — Multi-Stage Implicit Build

The application Dockerfile uses **best practices** for Python containerization:

```
FROM python:3.11-slim

LABEL org.opencontainers.image.source="https://github.com/aminghuf/URL_shortner"

LABEL org.opencontainers.image.description="Scalable URL Shortener with Distributed Orchestration"
LABEL org.opencontainers.image.version="1.0.0"

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
ENV FLASK_ENV=production

WORKDIR /app

# Install system dependencies for psycpg2
RUN apt-get update \
    && apt-get install -y --no-install-recommends gcc libpq-dev build-essential curl \
    && rm -rf /var/lib/apt/lists/*

# Install Python dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY . .

# Create non-root user
RUN addgroup --system app && adduser --system --ingroup app app &&
chown -R app:app /app
USER app

EXPOSE 8000

HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
    CMD curl -f http://localhost:8000/api/health || exit 1

CMD ["gunicorn", "--bind", "0.0.0.0:8000", "--workers", "4", "--threads", "2", \
```

```
        "--timeout", "120", "--access-logfile", "-", "--error-logfile",
    "-", "app:app"]
```

Key design decisions:

- **Slim base image:** python:3.11-slim minimizes attack surface and image size.
- **Layer caching:** COPY requirements.txt and RUN pip install are separated from COPY . to leverage Docker build cache — dependencies are only rebuilt when requirements.txt changes.
- **Non-root user:** The app user is created to run the application, enforcing least-privilege security.
- **HEALTHCHECK:** The Docker HEALTHCHECK instruction runs `curl -f http://localhost:8000/api/health` every 30 seconds with a 10-second startup grace period.
- **Gunicorn with workers & threads:** Four Gunicorn worker processes each handle two threads, providing up to 8 concurrent request handlers per container.

4.2 Nginx Dockerfile

A separate Nginx Dockerfile extends `nginx:1.25-alpine`, copying a custom `nginx.conf` with rate limiting, proxy settings, and security headers.

4.3 Docker Compose — Resource Constraints

The `docker-compose.yml` defines explicit **resource limits and reservations** for each service:

Service	CPU Limit	CPU Reservation	Memory Limit	Memory Reservation
Flask App	0.5	0.2	512M	256M
Nginx	0.2	0.1	128M	64M
PostgreSQL	0.5	0.2	512M	256M
Redis	0.3	0.1	256M	128M

These constraints enforce **cgroup-based CPU and memory isolation** — the Docker daemon translates these into Linux cgroup v2 parameters (`cpu.max`, `memory.max`, etc.), ensuring each container stays within its allocated resource budget.

4.4 Startup Dependencies

The Flask app waits for PostgreSQL to be healthy (depends_on.condition: service_healthy) before starting, preventing connection race conditions on startup.

5. Kubernetes Orchestration

The project includes a full set of Kubernetes manifests under `k8s/`, enabling deployment to any Kubernetes cluster.

5.1 Namespace (namespace.yaml)

A dedicated `url-shortener` namespace isolates all project resources:

```
apiVersion: v1
kind: Namespace
metadata:
  name: url-shortener
```

```
labels:
  app: url-shortener
  environment: production
```

5.2 ConfigMap (configmap.yaml)

Environment configuration is externalized via a ConfigMap:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: url-shortener-config
  namespace: url-shortener
data:
  DATABASE_URL: "postgresql://urlshortener:***@postgres-
service:5432/urlshortener"
  REDIS_URL: "redis://redis-service:6379/0"
  APP_NAME: "url-shortener"
  FLASK_ENV: "production"
```

The ConfigMap is injected into the deployment pod via `envFrom.configMapRef`, keeping credentials and configuration separate from the container image.

5.3 Deployment with 3 Replicas (deployment.yaml)

The Kubernetes Deployment manages 3 replicas of the Flask application container:

- **Container spec:** Runs `url-shortener:latest` image on port 8000 with environment variables from the ConfigMap.
- **Resource requests & limits:** Each pod requests 200m CPU / 256Mi memory and is capped at 500m CPU / 512Mi memory.
- **Three health probes:**
 - **Liveness probe:** HTTP GET `/api/health` on port 8000, every 15 seconds, failure threshold 3. Restarts the container if the app is unresponsive.
 - **Readiness probe:** HTTP GET `/api/health` every 10 seconds, failure threshold 2. Removes the pod from Service endpoints if not ready.
 - **Startup probe:** HTTP GET `/api/health` every 5 seconds, failure threshold 30. Gives the app up to 150 seconds to start before liveness probes kick in — essential for cold-start database migrations.

5.4 Horizontal Pod Autoscaler (hpa.yaml)

The HPA enables **elastic scaling** based on CPU utilization:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: url-shortener-hpa
  namespace: url-shortener
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: url-shortener
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
```



```
target:
  type: Utilization
  averageUtilization: 70
```

- **Minimum replicas:** 2 (ensures baseline availability and redundancy).
- **Maximum replicas:** 10 (burst capacity for traffic spikes).
- **Target CPU:** 70% average utilization across all pods. When aggregate CPU exceeds 70%, the HPA scales up; when it falls below, it scales down (after a cooldown period).

5.5 Service (service.yaml)

A ClusterIP Service exposes the deployment internally:

```
apiVersion: v1
kind: Service
metadata:
  name: url-shortener-service
  namespace: url-shortener
spec:
  type: ClusterIP
  selector:
    app: url-shortener
  ports:
    - name: http
      port: 80
      targetPort: 8000
      protocol: TCP
```

The Service maps port 80 to the container's port 8000 and load-balances across all healthy pod replicas.

5.6 Ingress (ingress.yaml)

An NGINX Ingress Controller routes external traffic to the Service:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: url-shortener-ingress
  namespace: url-shortener
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
    nginx.ingress.kubernetes.io/ssl-redirect: "false"
spec:
  ingressClassName: nginx
  rules:
    - host: url-shortener.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: url-shortener-service
                port:
                  number: 80
```

This routes all requests for `url-shortener.local` to the ClusterIP Service, enabling clean URL-based routing without port exposure.

6. CI/CD Pipeline

6.1 GitHub Actions Workflow

The CI/CD pipeline is defined in `.github/workflows/test.yml` with three stages:

Stage 1: Test & Build (test job)

Runs on `ubuntu-latest` with Python 3.11:

1. **Checkout code** — `actions/checkout@v4`
2. **Set up Python** — `actions/setup-python@v5` with Python 3.11
3. **Install dependencies** — `pip install -r requirements.txt`
4. **Run tests** — `pytest` executes the test suite
5. **Build Docker image** — `docker build -t url-shortener:test .`
6. **Login to Docker Hub** — uses GitHub Secrets (`DOCKER_USERNAME`, `DOCKER_PASSWORD`)
7. **Tag and push** — pushes both `:latest` and `:${{ github.sha }}` tags to Docker Hub

Stage 2: Deploy (deploy job)

Runs after the test job completes successfully:

1. **SSH into VPS** — uses `appleboy/ssh-action@v1.0.3` with VPS credentials from GitHub Secrets (`VPS_HOST`, `VPS_USER`, `VPS_SSH_KEY`)
2. **Stop and remove** the old container: `docker stop url-shortener && docker rm url-shortener`
3. **Pull latest image** from Docker Hub
4. **Run new container** with `docker run -d -p 5000:5000 --name url-shortener <image>:latest`
5. **Health check** — `curl -f http://127.0.0.1:5000/health` verifies the deployment succeeded

6.2 Self-Hosted Webhook Pattern

The SSH-based deploy in the GitHub Actions workflow acts as a **self-hosted webhook deployment** — the CI server connects to the production VPS over SSH and executes a rolling update script. This pattern is common in hybrid cloud/on-premise setups where direct Kubernetes registry webhooks are not available.

6.3 Secret Management

All sensitive credentials are stored as **GitHub Actions Secrets**:

Secret	Purpose
<code>DOCKER_USERNAME</code>	Docker Hub registry authentication
<code>DOCKER_PASSWORD</code>	Docker Hub registry authentication
<code>VPS_HOST</code>	Target deployment server hostname/IP
<code>VPS_USER</code>	SSH username for deployment server
<code>VPS_SSH_KEY</code>	Private SSH key for passwordless authentication

7. Performance & Scalability Considerations

7.1 Caching Strategy

- **Redis cache-aside** reduces database read load for the most frequent operation (redirects). Cache hit ratio is expected to

exceed 90% under steady-state traffic since short URLs are typically reused frequently within 24 hours.

- **Database connection pooling** (`pool_size=10`, `max_overflow=20`) minimizes connection overhead. Pool pre-ping ensures stale connections are detected and recycled before use.

7.2 Concurrency Model

- **Gunicorn with 4 workers × 2 threads** provides up to 8 concurrent request handlers per container. For CPU-bound tasks (URL validation, code generation), multiple workers exploit multi-core hosts. For I/O-bound tasks (database queries, Redis lookups), threading improves throughput.
- **ThreadPoolExecutor** (configurable via `BULK_IMPORT_WORKERS`) enables parallel processing of bulk imports. Chunked processing prevents a single large file from blocking the entire service.

7.3 Horizontal Scaling

- **Kubernetes HPA** automatically adjusts the number of pod replicas (2-10) based on CPU utilization.
- **Stateless application design** — all persistent state lives in PostgreSQL and Redis, so any pod can handle any request. This enables true horizontal scalability.
- **Nginx as load balancer** distributes traffic across replicas in the Docker Compose deployment mode.

7.4 Rate Limiting and Throttling

- **Two-layer rate limiting** (application + Nginx) protects against DoS attacks and abusive clients.
- **Nginx shared memory zone** can track approximately 160,000 distinct IPs in 10MB of RAM.

7.5 I/O Optimization

- **Batch inserts** (`bulk_insert_mappings`) instead of individual `session.add()` calls for bulk imports. This reduces the number of SQL round-trips from $O(n)$ to $O(1)$.
- **Redis pipelining** for bulk cache population — all SETEX commands are buffered and sent in one network round-trip.
- **Connection timeout guards:** Redis client configured with `socket_connect_timeout=2` and `socket_timeout=2` to prevent hanging on unavailable services.

7.6 Graceful Degradation

- Redis is **non-critical** — if unavailable, the app falls back to database-only mode.
- PostgreSQL unavailability returns 503 with a diagnostic message.
- Bulk import failures roll back cleanly via `db.session.rollback()`.

8. Virtualization Concepts Demonstrated

This project serves as a practical demonstration of the core virtualization concepts covered in the Virtualization Systems course:

8.1 Process Isolation (Namespaces)

Namespace Type	Demonstration
----------------	---------------

PID namespace	Each container has an isolated process tree. Gunicorn master (PID 1) inside the Flask container cannot see PostgreSQL or Redis processes.
Mount namespace	Each container has its own filesystem. The Flask container sees /app; PostgreSQL sees /var/lib/postgresql/data from the volume mount.
Network namespace	Each container gets its own IP address on the Docker bridge network. Inter-service communication uses DNS-based service discovery.
User namespace	The application runs as the non-root app user, mitigating privilege escalation risks.
UTS namespace	Each container can have its own hostname, independent of the host system.

8.2 Resource Constraints (cgroups v2)

Docker translates the `deploy.resources` block from `docker-compose.yml` into Linux cgroup v2 parameters:

- **cpu.max:** Enforces CPU limits (e.g., 50000 100000 for 0.5 CPU). The container cannot exceed its allocated CPU quota even if the host has idle cores.
- **memory.max:** Enforces memory limits (e.g., 512M for the Flask app). The OOM killer terminates the container if it exceeds this limit.
- **memory.min / memory.low:** Guarantees minimum memory reservations (256M for the Flask app).

These cgroup constraints enable **performance isolation** — a memory leak in Redis cannot starve PostgreSQL, and a CPU spike in the Flask app cannot degrade Nginx's proxy responsiveness.

8.3 Concurrency & Parallelism

- **Gunicorn worker/thread model** demonstrates the trade-off between process-based (true parallelism for CPU-bound work) and thread-based (shared memory for I/O-bound work) concurrency.
- **ThreadPoolExecutor** demonstrates controlled parallelism for bulk processing tasks, where the pool size limits resource consumption.

8.4 I/O Optimization

- **Connection pooling** reduces the overhead of establishing database connections.
- **Cache-aside pattern** (Redis) reduces disk I/O on PostgreSQL by serving fast cache hits from memory.
- **Batch operations** (bulk insert, Redis pipeline) minimize network round-trips.
- **Asynchronous health checks** (Kubernetes probes, Docker HEALTHCHECK) decouple monitoring from application request handling.

8.5 Declarative Orchestration (Kubernetes)

- **Desired state management:** Kubernetes continuously reconciles

the actual state (3 replicas, HPA scaling) with the declared state in YAML manifests.

- **Self-healing:** Probes detect unhealthy pods; Kubernetes terminates and replaces them automatically.
- **Horizontal autoscaling:** The HPA watches aggregate CPU metrics and adjusts replica count within the 2–10 range, demonstrating elastic resource provisioning.
- **Declarative networking:** Services and Ingress define the network topology declaratively, with kube-proxy implementing the actual packet forwarding rules.

8.6 Infrastructure as Code

- Docker Compose, Kubernetes manifests, and the CI/CD workflow are all **version-controlled in Git**, enabling reproducibility, audit trails, and collaborative development.
 - The entire system can be provisioned from scratch with `docker-compose up -d` or `kubectl apply -f k8s/`.
-

9. Deployment Instructions

9.1 Prerequisites

- Docker Engine 24+ with Docker Compose plugin
- Python 3.11 (for local development)
- `kubectl` (for Kubernetes deployment)
- A Kubernetes cluster (Minikube, kind, or cloud-based)

9.2 Local Development with Docker Compose

```
# Clone the repository
git clone https://github.com/aminghuf/URL_shortner.git
cd URL_shortner

# Set environment variables (optional, defaults provided)
export POSTGRES_USER=urlshortener
export POSTGRES_PASSWORD=urlshortener_secret
export POSTGRES_DB=urlshortener

# Start all services
docker compose up -d

# Check service status
docker compose ps

# View logs
docker compose logs -f

# Access the application
curl http://localhost:80/api/health
curl http://localhost:80/
```

The application is available at `http://localhost/`.

9.3 Deploy to Kubernetes

```
# Create the namespace and all resources
kubectl apply -f k8s/namespace.yaml
kubectl apply -f k8s/configmap.yaml
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
kubectl apply -f k8s/hpa.yaml
```

```

kubectll apply -f k8s/ingress.yaml

# Verify deployment
kubectll -n url-shortener get all

# Watch pods
kubectll -n url-shortener get pods -w

# Check HPA status
kubectll -n url-shortener get hpa

# Port-forward for local testing (if no ingress controller)
kubectll -n url-shortener port-forward svc/url-shortener-service
8080:80

```

9.4 Building and Running Manually

```

# Build the Flask image
docker build -t url-shortener:latest .

# Build the Nginx image
docker build -t url-shortener-nginx:latest ./nginx

# Run with explicit Docker commands
docker network create url-shortener-net

docker run -d --name postgres --network url-shortener-net \
-e POSTGRES_USER=urlshortener \
-e POSTGRES_PASSWORD=urlshortener_secret \
-e POSTGRES_DB=urlshortener \
-v postgres_data:/var/lib/postgresql/data \
postgres:16-alpine

docker run -d --name redis --network url-shortener-net redis:7-
alpine

docker run -d --name url-shortener --network url-shortener-net \
-e
DATABASE_URL=postgresql://urlshortener:urlshortener_secret@postgres:5432/urlshortener
\
-e REDIS_URL=redis://redis:6379/0 \
-p 8000:8000 \
url-shortener:latest

```

9.5 API Usage Examples

```

# Shorten a URL
curl -X POST http://localhost:80/shorten \
-H "Content-Type: application/json" \
-d '{"url": "https://www.example.com/very/long/url"}'

# Response:
#
{"short_code": "aB3xYz", "short_url": "http://localhost/aB3xYz", "created": true}

# Use the short URL (redirects)
curl -L http://localhost/aB3xYz

# Get statistics
curl http://localhost/stats/aB3xYz

# Bulk import from file
curl -X POST http://localhost:80/bulk-import \
-F "file=@urls.txt"

```

```
# Health check
curl http://localhost:80/api/health
```

10. Conclusion

This project successfully demonstrates a **production-grade, scalable URL shortener** that leverages the full spectrum of virtualization technologies covered in the Virtualization Systems course.

Achievements

1. **Complete Microservice Architecture:** Four specialized services (Flask, Nginx, PostgreSQL, Redis) communicate over a Docker bridge network, each fulfilling a single responsibility with well-defined interfaces.
2. **OS-Level Virtualization Mastery:** Every service runs in a container with namespace isolation (PID, mount, network, user, UTS) and cgroup resource constraints (CPU shares, memory limits, reservations). The Dockerfile follows security best practices with non-root user execution and minimal base images.
3. **Kubernetes Orchestration:** Full set of declarative YAML manifests covering namespace isolation, ConfigMap-based configuration, deployment with 3 replicas, multi-probe health checking (liveness, readiness, startup), ClusterIP Service abstraction, NGINX Ingress routing, and HPA-driven horizontal autoscaling (2-10 replicas at 70% CPU target).
4. **Production CI/CD Pipeline:** GitHub Actions workflow automates testing, Docker image building, Docker Hub publishing, and SSH-based deployment to a VPS — demonstrating the full development-to-production lifecycle.
5. **Performance Optimizations:** Redis caching (cache-aside pattern, 24h TTL, pipelining), database connection pooling (pool_size=10, max_overflow=20), concurrent bulk imports (ThreadPoolExecutor, chunked processing), transaction-wrapped batch inserts, two-layer rate limiting (application + Nginx), and gzip compression.
6. **Resilience Patterns:** Graceful degradation when Redis is unavailable, database rollback on insert failures, container restart policies, health probes, and startup delay handling in Kubernetes.

Educational Value

This project serves as a **comprehensive case study** in applied virtualization, demonstrating how theoretical concepts from the course — Linux namespaces, cgroups, container isolation, orchestration, resource management, I/O optimization, and infrastructure as code — are implemented in a real-world, deployable system. Each architectural decision is traceable to a specific virtualization principle, making the project a demonstrative artifact for the Virtualization Systems curriculum.

Repository

The complete source code and documentation are available at:
https://github.com/aminghuf/URL_shortner

This deliverable was prepared for the Virtualization Systems course, taught by Prof. Maria Fazio and Dr. Maurizio Giacobbe, as the final project for student Seyedamin Ghazizadeh (Student ID: 569071).

Date: June 12, 2026